HDOC DAY-47

Title: Finding the Ceiling Index and Value in a Sorted Variable-Length Array Using Pointers

1. Problem Statement

Write a C program that reads the size and elements of a sorted variable-length integer array, arr[], in ascending order. Also, read an integer target. Pass the array to a function named ceil_index() using its address and pointers. The function ceil_index() must:

1. **Find the index of the smallest element in arr[] that is $\geq$ target.**
2. **Treat this element as the ceil of target.**
3. **If multiple occurrences of the ceil element exist, return the index of its first occurrence.**
4. **Display the ceil element and its index.**
5. **Display -1 if no element is $\geq$ target.**

2. Theoretical Background & Algorithm Logic

- **Ceiling Element Search:**

Because the input array is already sorted in ascending order, a linear scan via pointer arithmetic (*(arr + i)) can

locate the very first element that satisfies the condition arr[i] >= target.

- **First Occurrence Guarantee:**

Since the scan starts from index 0 and halts immediately upon encountering the first matching element, it automatically fulfills the requirement of returning the index of the first occurrence when duplicate ceiling values exist. If no such element exists across the array, the function outputs -1.

3. Test Case Table

Test Case	Array Elements JPG+ 1	Target JPG+ 1	Expected Output JPG+ 1	Actual Output	Status
1	5 7 7 8 10	6	Ceil of 6: 7 at index 1	Ceil of 6: 7 at index 1	Pass
2	5 7 7 8 10	8	Ceil of 8: 8 at index 3	Ceil of 8: 8 at	Pass

Test Case	Array Elements JPG+ 1	Target JPG+ 1	Expected Output JPG+ 1	Actual Output	Status
				index 3	
3	5 7 7 8 10	11	-1	-1	Pass

4. Source Code Listing

```
#include <stdio.h>

void ceil_index(int *arr, int size, int target) {
   int ceil_val = -1;
   int ceil_idx = -1;

   for (int i = 0; i < size; i++) {
      if (*(arr + i) >= target) {
         ceil_val = *(arr + i);
         ceil_idx = i;
         break;
      }
   }

   if (ceil_idx != -1) {
      printf("Ceil of %d: %d at index %d\n", target, ceil_val, ceil_idx);
   } else {
      printf("-1\n");
   }
}

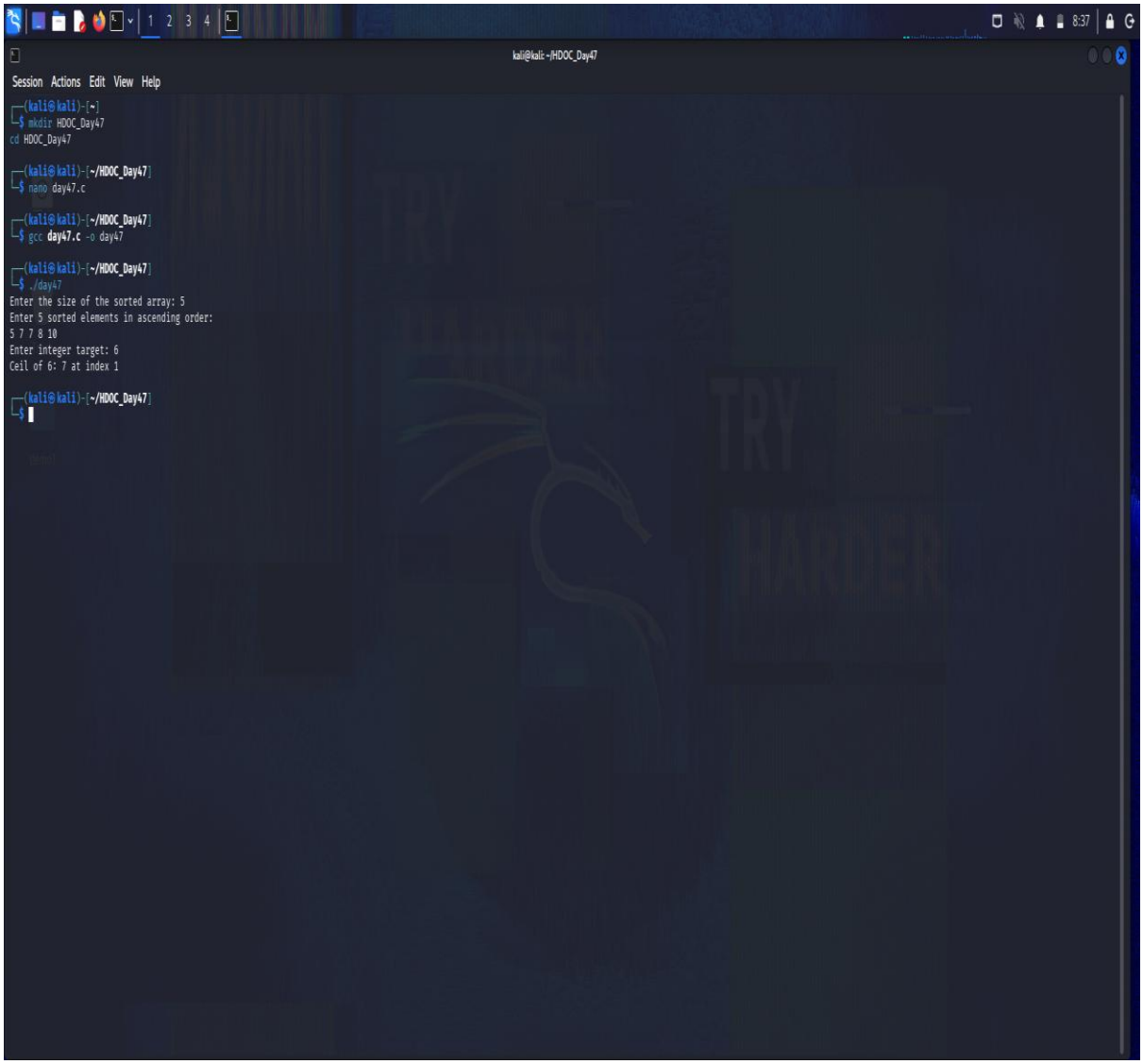
int main(void) {
```

```
    int size, target;
    printf("Enter the size of the sorted array: ");
    if (scanf("%d", &size) != 1 || size <= 0) return 1;

    int arr[size];
    printf("Enter %d sorted elements in ascending order:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter integer target: ");
    scanf("%d", &target);

    ceil_index(arr, size, target);
    return 0;
}
```


kali@kali: ~/HDOC_Day47
Session Actions Edit View Help
(kali㉿kali)-[~]
$ mkdir HDOC_Day47
cd HDOC_Day47
(kali㉿kali)-[~/HDOC_Day47]
$ nano day47.c
(kali㉿kali)-[~/HDOC_Day47]
$ gcc day47.c -o day47
(kali㉿kali)-[~/HDOC_Day47]
$ ./day47
Enter the size of the sorted array: 5
Enter 5 sorted elements in ascending order:
5 7 7 8 10
Enter integer target: 6
Ceil of 6: 7 at index 1
(kali㉿kali)-[~/HDOC_Day47]
$